| DS | **Session** | Queue | **Question Information** | Level 1 Challenge 54 |
|---|---|---|---|---|

**Question description**

Joe root is a Placement trainer. he is working as CDC trainer in reputed institution that during training the youngsters are struggling in queue concept.

Joe root usually gives interesting problems to the students to make them love the DS.

One such day Joe root provided to the final year students to solve the task such that, Queue implementation with arrays as using linked list for implementing queue and delete an element from the queue using linked list concept.

Queue data structures work on the FIFO architecture so the element that has entered first in the list will go out from the list first.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

**Function Description**

## enqueue(data)

- Build a new node with given data.
- Check if the queue is empty or not.
- If queue is empty then assign new node to front and rear.
- Else make next of rear as new node and rear as new node.

## dequeue()

- Check if queue is empty or not.
- If queue is empty then dequeue is not possible.
- Else store front in temp
- And make next of front as front.

## print()

- Check if there is some data in the queue or not.
- If the queue is empty print “No data in the queue.”
- Else define a node pointer and initialize it with front.
- Print data of node pointer until the next of node pointer becomes NULL.

**Constraints:**

0<size <100

0<data<1000

**Input format:**

First line indicates the size of the queue

Second line indicates the elements of the queue.

**Output Format:**

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

## Logical Test Cases

### Test Case 1

INPUT (STDIN)

```
5
9 1 5 3 5
```

EXPECTED OUTPUT

```
9 1 5 3 5
1 5 3 5
```

### Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
```

## Mandatory Test Cases

### Test Case 1

KEYWORD

```
void enqueue(int d)
```

### Test Case 2

KEYWORD

```
struct node* new_n;
```

### Test Case 3

KEYWORD

```
new_n = (struct node*)malloc(sizeof(struct node));
```

### Test Case 4

KEYWORD

```
struct node *rear = NULL;
```

### Test Case 5

KEYWORD

```
struct node *front = NULL;
```

## Complexity Test Cases

### Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

### Test Case 2

TOKEN COUNT

```
300
```

### Test Case 3

NLOC

```
70
```

# Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

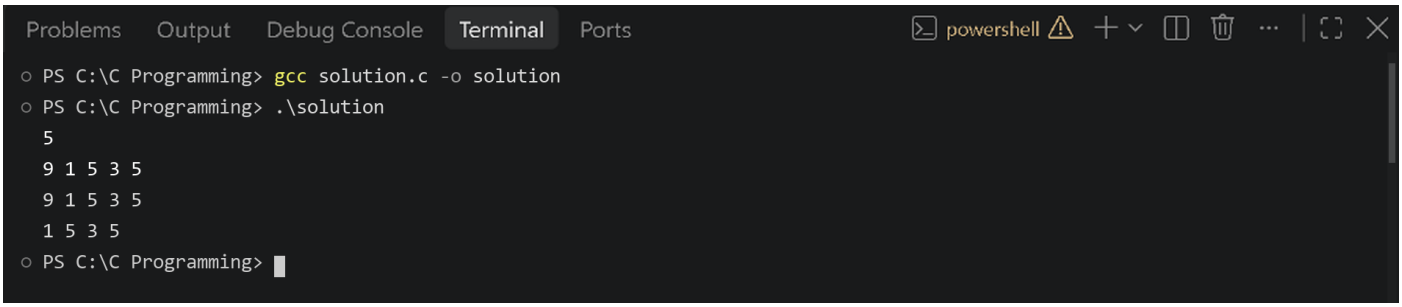
void enqueue(int d) {
    struct node *new_n;
    new_n = (struct node*)malloc(sizeof(struct node));
    new_n->data = d;
    new_n->next = NULL;
    if (front == NULL) {
        front = rear = new_n;
    } else {
        rear->next = new_n;
        rear = new_n;
    }
}

void dequeue() {
    if (front != NULL) {
        struct node *t = front;
        front = front->next;
        free(t);
    }
}

void print() {
    struct node *t = front;
    while (t != NULL) {
        printf("%d%s", t->data, t->next ? " " : "");
        t = t->next;
    }
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        enqueue(x);
    }
    print();
    dequeue();
    print();
    return 0;
}
```

## Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
PS C:\C Programming>